

Song Popularity and Recommendation Algorithm - Analysis and Predictions

by Andres Silva-Castellanos and Atul Triplicane

Introduction

We have decided to select the music analysis topic from the original project spec. Overall, we want to find out what song attributes correlate to popularity among several variables such as metadata like rhythm, energy, etc as well as genre. Then, we are going to apply all our results/findings from the initial analysis to make some kind of song recommendation algorithm tailored to a user's tastes, considering popularity as a factor.

As mentioned above, the overall question we are asking is, "How can we cater new recommendations to a listener based on the attributes of the songs they listen to?" In addition, we are going to answer the following questions to gather the necessary data to accomplish this:

- Which attributes correlate with song popularity?
- How does the size of the dataset affect efficiency in finding song similarity?
- What methods can be employed to find the best recommendations?

Related Work

The biggest example of a similar system that exists is called "DJ". This is an additional feature that was recently added to Spotify. This AI tool provides the listener with a queue of songs that match the listener's tastes. Every five songs, the AI switches the genre or tone of the music to something different (Mangla). Another example of a similar system that already exists is an analysis conducted by Peter Dola about the statistics behind how music has changed in the past 100 years. This analysis provides us with various attributes that are used such as Instrumental, Duration, Popularity, etc, and overall, different ways we can use a music dataset to find answers to questions we want to ask. We also used an existing Kaggle notebook (6) to help guide us through the process of how to set

up DataFrames for using cosine similarity. However, what we provided in this notebook was that we are comparing this method of finding recommendations to a clustering method that does the same thing, and comparing results from both a smaller and a large dataset.

Methods

We decided to work with two datasets. The first dataset is much larger, consisting of 114,000 songs from Spotify. The second dataset is smaller, consisting of 953 songs. However, this dataset is more specific because it only contains the most popular Spotify songs in 2023. Both datasets had different attributes that determine the popularity of a song. For example, the first dataset has attributes such as genre and popularity scale that the second dataset didn't have. Another example is that the second dataset has additional attributes such as bpm, playlist counts, and chart counts.

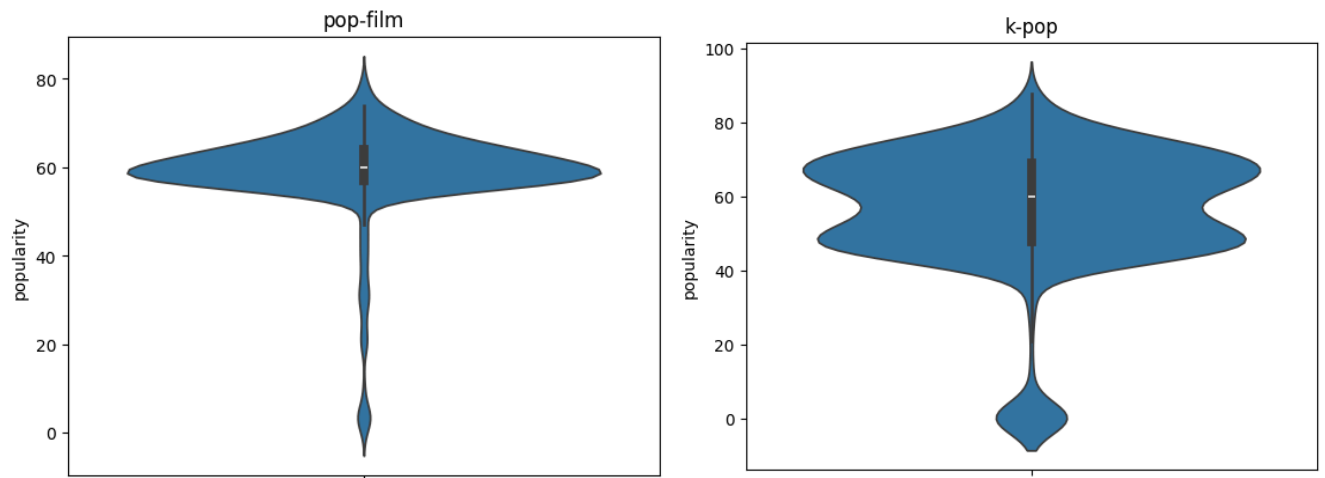
- We started by dissecting the first dataset.
 - Used Pearson's and Spearman's Correlation Coefficients to find out how popularity correlates to all the numerical attributes.
 - Found popularity correlations among non-numerical-based attributes based on the mean.
 - Made Violin Plots to find the distributions of the top 5 most popular genres in the dataset
 - Repeating the last 3 points, but with only the top 5,000 popular tracks
 - Compared the top genres in the original dataset vs the top genres in the top 5000 tracks
 - Found the best value of n based on the silhouette score for clustering
 - Used K-means clustering to find subsets of tracks and found the correlation between features and a song's cluster
 - Trained models for each cluster, and found belonging cluster using K-Nearest Neighbor

- Used cosine similarity on any input song's features to find the most similar songs.
- Made algorithms to make music recommendations based on the given songs, using either our clusters or cosine similarity scores.
- We then dissected the second dataset
 - Since this dataset didn't have a popularity scale, we devised an algorithm to come up with a popularity scale using the streams, charts, and playlists
 - Normalized the data so that the popularity scale would be between the numbers 0-100
 - Used Pearson's and Spearman's Correlation Coefficients to find out how popularity correlates to all the numerical attributes.
 - Found popularity correlations among non-numerical-based attributes based on the mean.
 - Found the best value of n based on the silhouette score for clustering
 - Used K-means clustering to find subsets of tracks and found the correlation between features and a song's cluster
 - Trained models for each cluster, and found belonging cluster using K-Nearest Neighbor
 - Used cosine similarity on any input song's features to find the most similar songs.
 - Made algorithms to make music recommendations based on the given songs, using our cosine similarity scores.

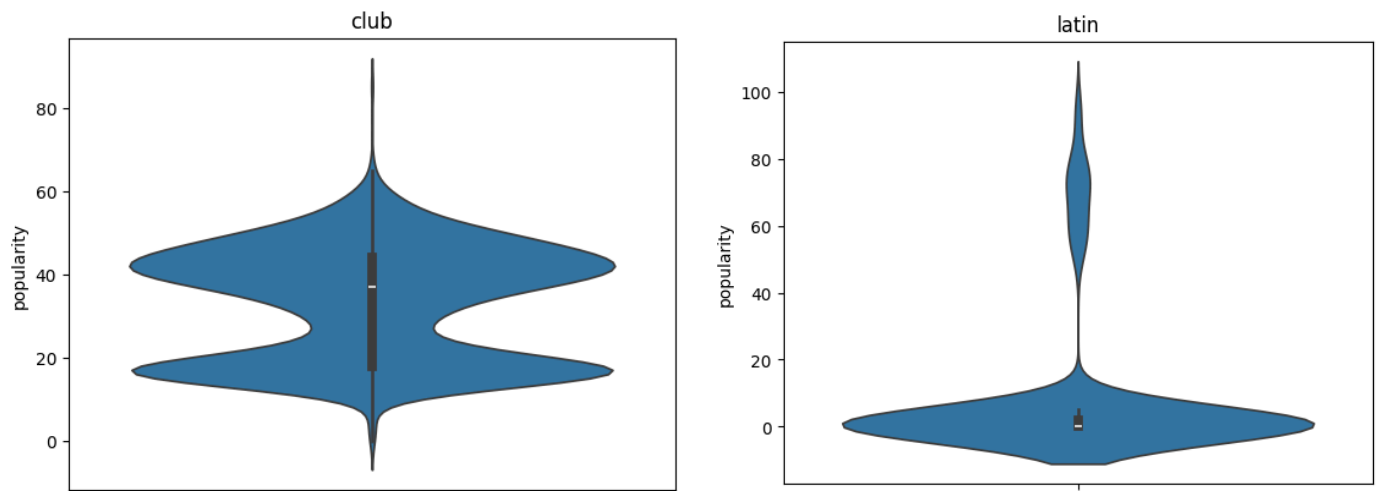
Results

First, when trying to correlate between any attribute and popularity, we concluded that there was no significant correlation for any one attribute. All the resulting coefficients did not exceed even 0.1 or -0.1, for either dataset. However, we were able to measure the popularity of genres with the dataset that includes genre as an attribute. Among the top 5 most popular genres, we plotted the distributions

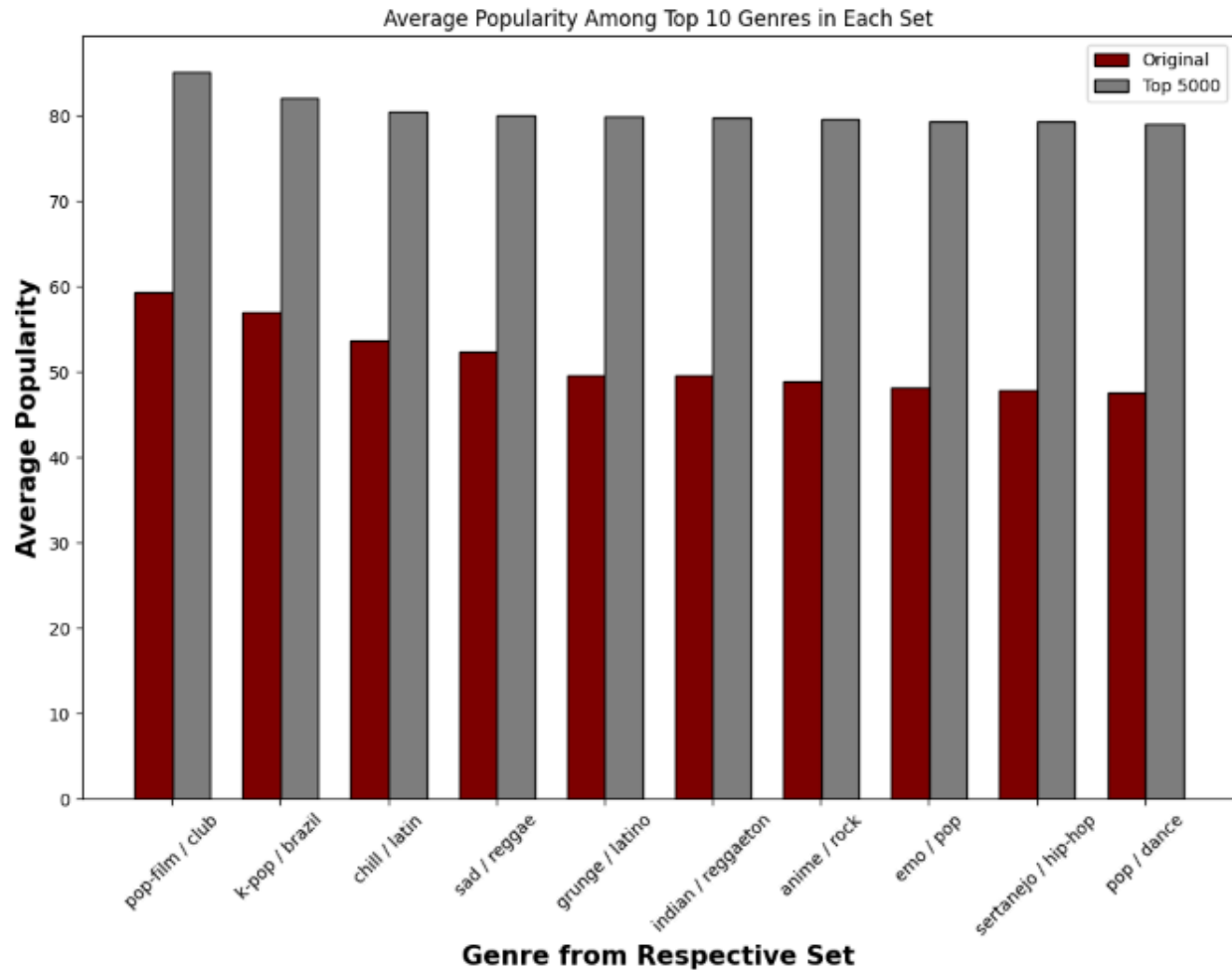
using violin plots. Here are the plots for the top 2:



From these plots, we can see that the means of all the songs of those genres are close to 60 with different distributions on each side. Pop-film songs are consistently more popular than K-pop songs. However, when we did the same analysis for the top 5000 most popular songs in that dataset, we were surprised to find that almost all of the top genres were different. Here are two such plots:

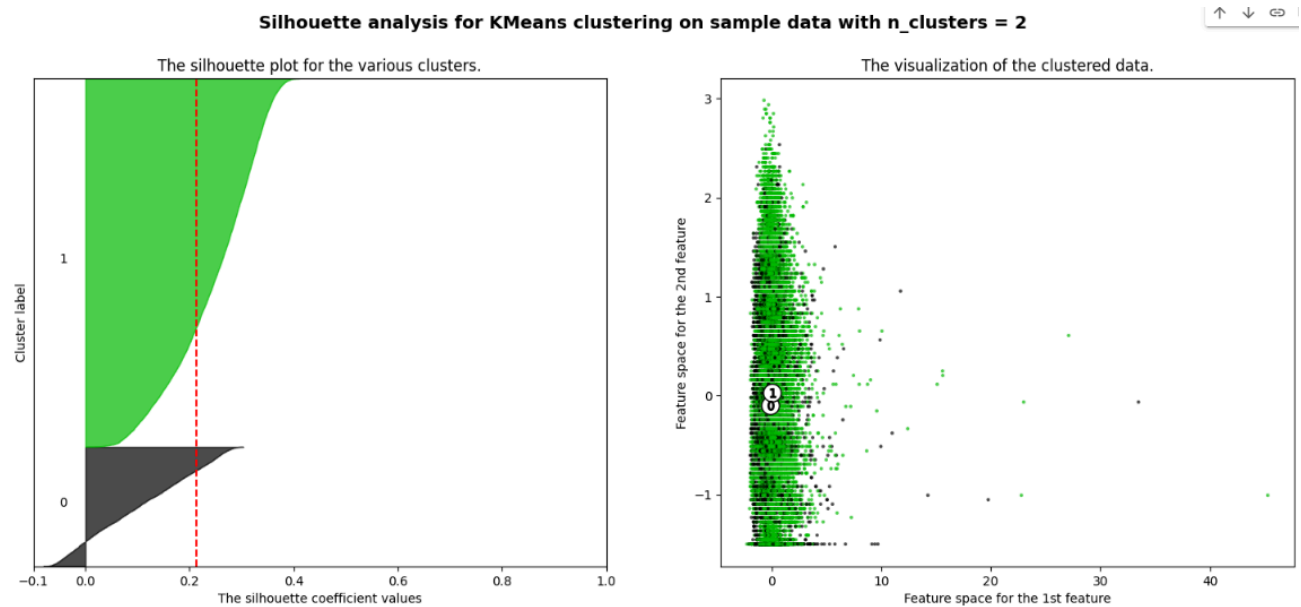


We were especially surprised at the distribution of Latin songs because most had very low popularity. However, there are still many songs higher up in popularity than the median which would account for all the Latin songs occurring in the top 5000. To conclude our popularity analysis, we found that the songs in the top 5000 had, on average, higher popularity than songs in the original dataset, which makes perfect sense.



Now we started to do our clustering model, but first, we needed to find the best number of clusters to use, so we did a silhouette score test first. Unfortunately, we found that there was no silhouette score close to 1 and thus, no clear clustering with any number of clusters after trying with $n=2$ and various other values up to $n=20$. Although, $n=2$ had the best score of ~ 0.21 . This visual demonstrates the concentration of points in each cluster and the scatter plot with the respective clusters based on colors. The red line represents the average

silhouette score among all the clusters.



We also found that the feature that impacted the different clusterings the most was duration, followed by instrumentalness. Duration makes sense because even when the values are normalized, the scale for the duration attribute is larger than the other ones. Then, we found that our KNN model predicted the correct cluster every point in the test set belonged to with an accuracy of 99% for the large dataset and ~75% for the small one. We assume that the high accuracy of the former is due to the sheer size of the set.

Finally, we used cosine similarity to find similarity between every pair of songs in the dataset based on the datasets' respective attributes (one large one including genre and a smaller one not including genre). For the song "Save Your Tears" by The Weeknd, these were the songs that our recommendation algorithm produced

with their respective similarity scores to the input song:

Recommended Songs:

Title: Like Crazy
Artist(s): Jimin
Similarity: 0.9768056860350066

Title: Dance The Night (From Barbie The Album)
Artist(s): Dua Lipa
Similarity: 0.9735646179763272

Title: Levitating
Artist(s): Dua Lipa
Similarity: 0.970246165826561

Title: About Damn Time
Artist(s): Lizzo
Similarity: 0.9698895141043774

Title: Ghost
Artist(s): Justin Bieber
Similarity: 0.9690661464685248

Even without the similarity scores, just from looking at the songs, we can tell that they have very similar sounds or are at least in the same genre. This is a significant find because this dataset (smaller) did not use genre as a factor when computing the cosine similarity vectors. Overall, we found that the cosine similarity algorithm is much more concrete, reliable, and reproducible than the clustering algorithm we came up with. As for the size of the datasets, we didn't find any significance in the results of our recommendation algorithms to suggest there is any overall effect on the efficiency of the algorithm (besides the runtime).

Conclusions

In our analysis, we explored what features in a song contribute to a song's popularity. Then we made models based on clustering (Silhouette Score, K-means, KNN) and cosine similarity to determine if one method was more efficient than the other. We found that our recommendation algorithm using cosine similarity gave more reliable results. This is likely because a song's vector is specific and takes every feature into account. We can conclude that there is a

large variety of attributes that are used to determine song popularity, the size of the dataset does not affect the reliability of the algorithm, and the best method that can be employed to find the song recommendations is cosine similarity. If we had more time, we would have added other features such as filtering, creating new playlists, and even using past musical statistics to predict new music trends.

References

Here are all the datasets and references we used to gather information and come up with our questions, as well as guides for writing specific parts of our models:

Datasets:

1. Maharshi Pandya. (2022).  Spotify Tracks Dataset [Data set]. Kaggle. <https://doi.org/10.34740/KAGGLE/DSV/4372070>
2. [Most Streamed Spotify Songs 2023 \(kaggle.com\)](#)

Readings / Aid:

3. Dola, Peter, “Exploratory Analysis”, RPubS, <https://rpubs.com/PeterDola/SpotifyTracks>
4. Mangla, Puneet, “Spotify Music Recommendation Systems”, Pyimagesearch, <https://pyimagesearch.com/2023/10/30/spotify-music-recommendation-systems/>
5. Silhouette Scores and Graphs: https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html#sphx-glr-auto-examples-cluster-plot-kmeans-silhouette-analysis-py
6. Setting up DataFrames for cosine similarity: <https://www.kaggle.com/code/akbareza/simple-music-recommendation-system>